

UNIVERSIDADE DE MOGI DAS CRUZES

ERICK SANTOS BARBOSA

GABRIEL AUGUSTIN FERNANDEZ ALVES

PEDRO HENRIQUE HARADA PECEGUEIRO

SELIEL ELIAS NUNES FILHO

JURIS-AI: PLATAFORMA DE APOIO AO ESTUDO JURÍDICO

MOGI DAS CRUZES

2026

JURIS-AI: PLATAFORMA DE APOIO AO ESTUDO JURÍDICO

Trabalho acadêmico do curso de Engenharia de Software, apresentado à Universidade de Mogi das Cruzes como entrega técnico-científica do sistema JurisAI. Este documento consolida a visão geral do sistema, sua arquitetura, fluxos de autenticação e dados, gestão de credenciais, cuidados de segurança aplicados ao produto, testes realizados e referências utilizadas.

Orientador: Prof. Fabiano Bezerra Menegidio

Sumário

1. Sistema Juris-AI.....	4
2. Visão geral do sistema.....	5
3. Arquitetura do sistema	7
4. Fluxos de autenticação e dados	9
4.1 Fluxo de cadastro.....	9
4.2 Fluxo de login	9
4.3 Fluxo de autenticação em duas etapas	10
4.4 Fluxo de validação do token JWT	10
4.5 Fluxo de logout	11
4.6 Fluxo de recuperação de senha	11
5. Gestão de credenciais	12
6. Uso de criptografia.....	13
7. Identificação dos ativos do sistema	14
8. Identificação de ameaças e vulnerabilidades	15
9. Associação riscos e contramedidas.....	16
10. Testes de segurança realizados	17
10.1 Resultados dos testes	18
Uso de artigos científicos e/ou normas técnicas.....	19
Referências.....	20

1. Sistema Juris-AI

Este documento apresenta a documentação técnico-científica do sistema Juris-AI, com foco no funcionamento geral da solução, nas tecnologias usadas, na organização do projeto, nos fluxos principais e nos cuidados de segurança e privacidade adotados.

O Juris-AI foi pensado como uma plataforma de apoio ao estudo e à prática jurídica. A ideia é ajudar estudantes de Direito, candidatos à OAB e jovens profissionais a organizar informações, estudar documentos, treinar argumentos e acessar recursos de forma segura. Dentro desse projeto, temos uma base de autenticação e controle de usuários, ou seja, a parte que cuida de cadastro, login, perfil, autenticação em duas etapas, recuperação de senha, área administrativa e proteção dos dados básicos do usuário.

Assim, o foco deste documento é a segurança do nosso sistema, mas sempre relacionada ao funcionamento do Juris-AI como produto: quem usa, quais dados circulam, como o usuário entra, como o sistema protege o acesso e como essa base permite que os módulos jurídicos e de inteligência artificial possam evoluir com mais responsabilidade.

2. Visão geral do sistema

O Juris-AI é um sistema desenvolvido para aproximar estudantes e profissionais iniciantes da prática jurídica com apoio de tecnologia. O objetivo principal é oferecer um ambiente no qual o usuário consiga criar sua conta, acessar uma área personalizada, proteger seus dados e, futuramente, usar recursos de inteligência artificial voltados ao estudo de casos, leitura de documentos e treino de argumentação.

O sistema não foi pensado apenas como uma tela de login ou cadastro. Essa parte de autenticação é importante porque garante que cada usuário tenha acesso seguro à sua própria conta, mas ela faz parte de uma proposta maior: desenvolver uma plataforma jurídica educacional que ajude o estudante a organizar seus estudos, acessar recursos personalizados, praticar argumentação e, futuramente, utilizar ferramentas de inteligência artificial como apoio no aprendizado jurídico.

O auth-api funciona como o núcleo de autenticação do sistema. Em outras palavras, ele é responsável por responder perguntas importantes como: quem é o usuário, se ele pode entrar, quais dados pertencem a ele, se a conta está protegida, se a senha está correta e se o acesso deve ser permitido.

Nós estruturamos esse microsserviço porque qualquer sistema que lida com usuários precisa ter uma base confiável de identidade. Antes de o usuário utilizar recursos jurídicos ou acadêmicos, o sistema precisa permitir cadastro, login, proteção de sessão e gerenciamento de perfil. Essa camada de segurança não é o produto inteiro, mas é o que permite que o restante do Juris-AI seja usado com mais confiança.

O sistema resolve um problema prático: muitos projetos acadêmicos focam apenas na funcionalidade principal e deixam autenticação, segurança e privacidade como algo secundário. No Juris-AI, nós tratamos essa parte como base do projeto, porque uma ferramenta jurídica precisa cuidar bem de identidade, permissões e dados do usuário.

O escopo atual do sistema inclui:

- Cadastro de usuários com e-mail, senha e aceite LGPD.
- Login com autenticação por e-mail e senha.
- Proteção de senha usando hash.
- Autenticação em duas etapas com aplicativo autenticador.
- Recuperação e redefinição de senha.
- Perfil do usuário com dados acadêmicos/profissionais.
- Exportação e exclusão de dados do usuário.
- Área administrativa para gerenciamento de usuários.
- Logs de auditoria para registrar ações importantes.
- Frontend com telas de login, cadastro, dashboard, perfil, 2FA e administração.
- Base preparada para expansão futura de módulos jurídicos, como análise de documentos, estudo de casos e apoio por IA.

As tecnologias principais usadas foram:

Tecnologia	Onde usamos	Por que escolhemos
FastAPI	Backend/API	Facilita a criação de APIs em Python e gera documentação automática
Python	Linguagem principal	Linguagem simples, produtiva e adequada para protótipos acadêmicos
SQLAlchemy	Acesso ao banco	Ajuda a trabalhar com banco usando modelos em Python
MySQL	Banco de dados	Armazena usuários, tokens, permissões e logs
Pydantic	Validação de dados	Garante que os dados enviados para a API estejam corretos
bcrypt	Senhas	Evita armazenar senha em texto puro
JWT	Sessões	Permite controlar o acesso do usuário autenticado
PyOTP	2FA	Permite autenticação em duas etapas
HTML, CSS e JavaScript	Frontend	Interface do usuário

3. Arquitetura do sistema

A arquitetura do sistema foi organizada em camadas simples. Essa escolha facilita a manutenção, a leitura do código e a divisão de responsabilidades entre as partes do projeto. Em vez de concentrar tudo em um único arquivo, nós separamos o sistema em frontend, rotas, serviços, modelos, configuração e banco de dados.

No desenho atual, o sistema entregue está concentrado na autenticação, no perfil do usuário e na administração. Porém, essa arquitetura já deixa espaço para os módulos principais do Juris-AI, como dashboard jurídico, análise de documentos, estudo guiado e recursos de inteligência artificial.

A[Usuário] --> B[Frontend HTML, CSS e JavaScript]

B --> C[API FastAPI]

C --> D[Rotas / Routers]

D --> E[Serviços do sistema]

E --> F[Camada de segurança e configuração]

E --> G[Banco de dados MySQL]

E --> H[Serviço de e-mail]

C --> I[Módulos jurídicos futuros]

I --> I1[Análise de documentos]

I --> I2[Treino de argumentação]

I --> I3[Apoio com IA]

D --> D1[auth.py]

D --> D2[mfa.py]

D --> D3[user.py]

D --> D4[admin.py]

D --> D5[password_reset.py]

E --> E1[AuthService]

E --> E2[MFAService]

E --> E3[UserService]

E --> E4>PasswordService]

E --> E5[AuditService]

Na prática, o usuário acessa as telas do frontend, como login, cadastro ou perfil. Essas telas enviam requisições para a API. A API recebe os dados, valida a entrada, chama os serviços responsáveis e salva ou consulta informações no banco de dados. Quando os módulos jurídicos forem evoluídos, eles deverão reutilizar essa mesma base de autenticação para saber quem está usando o sistema e quais dados pertencem a cada usuário.

Divisão principal:

Camada	Responsabilidade
Frontend	Exibir telas e enviar dados para a API
Routers	Receber as requisições HTTP e encaminhar para os serviços
Services	Executar as regras de negócio
Core	Guardar funções de segurança, criptografia, configuração e banco
Models	Representar as tabelas do banco
Schemas	Validar os dados de entrada e saída
Banco de dados	Armazenar usuários, tokens, logs e consentimentos

Essa organização ajudou a deixar o projeto mais compreensível. Por exemplo, a rota de login não precisa conter toda a regra de autenticação; ela chama o **AuthService**, que concentra a lógica. Isso torna o código mais limpo e mais fácil de testar.

Também usamos uma arquitetura compatível com evolução futura. O Juris-AI poderá receber novos módulos, como análise de documentos, histórico de estudos e integração com IA, sem precisar misturar essas funcionalidades diretamente dentro do código de autenticação.

4. Fluxos de autenticação e dados

Os fluxos documentados mostram como os dados caminham pelo sistema desde o momento em que o usuário interage com a tela até a resposta final da API. Como o repositório atual representa a base de usuários do Juris-AI, os fluxos principais são de cadastro, login, sessão, perfil, 2FA, logout e recuperação de senha. Esses fluxos não existem isolados: eles são o caminho para que o usuário consiga chegar com segurança aos recursos do sistema jurídico.

4.1 Fluxo de cadastro

No cadastro, o usuário informa dados básicos como e-mail, senha, nome, universidade, semestre, tipo de perfil, especialidade jurídica e aceite da LGPD. Esses dados ajudam o sistema a criar uma conta personalizada e também permitem que, no futuro, o JurisAI adapte a experiência ao perfil do usuário, por exemplo estudante, advogado ou outro tipo de perfil.

actor U as Usuário
participant F as Frontend
participant A as API
participant S as AuthService
participant B as Banco de Dados

U->>F: Preenche formulário de cadastro
F->>A: Envia dados para /auth/register
A->>A: Valida os dados
A->>S: Solicita criação do usuário
S->>S: Gera hash da senha
S->>B: Salva usuário e consentimento LGPD
S-->>A: Retorna usuário criado
A-->>F: Retorna token e dados básicos

Esse fluxo é importante porque cria a identidade inicial do usuário no sistema. Também é nesse momento que registramos o aceite da LGPD, indicando que o usuário concordou com o uso dos dados para autenticação, segurança da conta e funcionamento dos recursos do JurisAI.

4.2 Fluxo de login

No login, o usuário informa e-mail e senha. A API verifica se o e-mail existe, se a conta não está bloqueada e se a senha está correta. Se tudo estiver certo, o sistema gera um token JWT para representar a sessão do usuário.

Passos principais:

- O usuário informa e-mail e senha.
- A API procura o usuário no banco.
- A senha digitada é comparada com o hash salvo.
- Se houver muitas tentativas erradas, a conta pode ser bloqueada temporariamente.
- Se o login for válido, a API retorna um token de acesso.
- O frontend usa esse token para acessar áreas protegidas.

4.3 Fluxo de autenticação em duas etapas

A autenticação em duas etapas foi incluída para aumentar a segurança da conta. O usuário pode configurar um aplicativo autenticador, como Google Authenticator, e depois precisa informar um código temporário ao fazer login.

O fluxo funciona assim:

- O usuário logado solicita a configuração do 2FA.
- O sistema gera um segredo TOTP.
- O segredo é convertido em QR Code para o aplicativo autenticador.
- O usuário confirma o código gerado no aplicativo.
- O sistema ativa o 2FA na conta.
- Nos próximos logins, além da senha, o usuário também informa o código.

Esse recurso é útil porque reduz o risco de invasão caso a senha seja descoberta. Mesmo assim, ele continua sendo simples para o usuário, pois depende apenas de um aplicativo autenticador.

4.4 Fluxo de validação do token JWT

Depois do login, o usuário recebe um token JWT. Esse token é enviado pelo frontend em requisições para rotas protegidas, como perfil, dashboard ou área administrativa. Em uma evolução do sistema, esse mesmo controle será usado para proteger documentos, históricos de estudo, casos enviados e respostas geradas pela IA.

Caminho da validação:

- O frontend envia o token no cabeçalho `Authorization`.
- A API verifica se o token está na blacklist.
- Se não estiver bloqueado, a API valida a assinatura do token.
- A API extrai o identificador do usuário.
- O usuário é buscado no banco.
- Se tudo estiver correto, a rota é liberada.

Esse fluxo evita que qualquer pessoa acesse dados sem estar autenticada. Também permite que o backend saiba qual usuário está fazendo a requisição.

4.5 Fluxo de logout

No logout, o sistema registra o token em uma blacklist. Isso significa que, mesmo que o token ainda não tenha expirado, ele passa a ser recusado pela API.

Esse comportamento é importante porque melhora o controle de sessão. Sem blacklist, o token continuaria válido até o tempo de expiração.

4.6 Fluxo de recuperação de senha

O sistema possui recursos para recuperação e redefinição de senha. A ideia é permitir que o usuário recupere o acesso sem comprometer a segurança da conta.

O fluxo planejado:

- O usuário solicita recuperação de senha.
- O sistema gera um token temporário.
- O token possui prazo de validade.
- O usuário cria uma nova senha.
- A senha nova é salva com hash.
- O token usado deixa de ser válido.

Esse fluxo ainda deve ser revisado antes de produção, porque recuperação de senha é uma das partes mais sensíveis de qualquer sistema de autenticação.

5. Gestão de credenciais

A gestão de credenciais trata de como o sistema cuida de informações sigilosas, como senhas, chaves de assinatura, dados de banco e credenciais de e-mail.

No JurisAI, essas informações não devem ficar escritas diretamente no código. Para isso, usamos variáveis de ambiente, normalmente configuradas em um arquivo `.env` no ambiente local ou no painel da plataforma de hospedagem.

As principais variáveis usadas são:

Variável	Finalidade
DATABASE_URL	Conexão com o banco de dados
SECRET_KEY	Assinatura dos tokens JWT
FERNET_KEY	Criptografia de informações sensíveis
ACCESS_TOKEN_EXPIRE_MINUTES	Tempo de duração do token
ENVIRONMENT	Define se o sistema está em desenvolvimento ou produção
SMTP_EMAIL	E-mail usado para recuperação de senha
SMTP_PASSWORD	Senha de aplicativo do e-mail

A senha do usuário não é salva diretamente no banco. Quando o usuário cria uma senha, o sistema transforma essa senha em um hash. Assim, mesmo que alguém visualize o banco de dados, não verá a senha original.

Também documentamos como cuidado importante que arquivos de exemplo não devem conter senhas reais. Caso alguma senha real tenha sido colocada por engano em `.env.example`, ela deve ser trocada imediatamente.

De forma simples, a regra adotada é: senha de usuário não fica em texto puro, chave secreta não fica no código e credencial de serviço deve ficar apenas no ambiente.

6. Uso de criptografia

O sistema usa mecanismos de proteção de dados em três pontos principais: senha, token de sessão e segredo do 2FA.

Recurso	Técnica usada	Explicação simples
Senha do usuário	bcrypt	Transforma a senha em um hash seguro
Sessão do usuário	JWT com HS256	Gera um token assinado para identificar o usuário
Segredo do 2FA	Fernet	Criptografa o segredo usado pelo aplicativo autenticador
Código 2FA	TOTP	Gera códigos temporários de autenticação
Recuperação de senha	Token aleatório	Cria um link ou código temporário para redefinição

O **bcrypt** foi escolhido porque é uma técnica comum para proteger senhas. Ele dificulta ataques em massa, pois foi feito para ser mais lento do que hashes simples.

O JWT é usado para que o sistema saiba que o usuário já fez login. O token possui tempo de expiração configurável. No arquivo de exemplo, esse tempo aparece como 30 minutos.

O TOTP é usado no 2FA. Ele gera códigos temporários, geralmente renovados a cada 30 segundos. Assim, mesmo que alguém descubra a senha, ainda precisa do código do aplicativo autenticador.

A criptografia com Fernet é usada para proteger o segredo do 2FA no banco de dados. Isso é importante porque esse segredo permite gerar os códigos temporários.

7. Identificação dos ativos do sistema

Ativos são elementos importantes do sistema que precisam ser protegidos ou bem administrados. No JurisAI, os ativos principais estão relacionados aos dados dos usuários, ao código-fonte, ao banco de dados e aos endpoints da API.

Ativo	Por que é importante
Dados dos usuários	Identificam os usuários e seus perfis
Senhas com hash	Protegem o acesso às contas
Tokens JWT	Representam sessões autenticadas
Segredo 2FA	Protege a segunda etapa de login
Banco de dados	Guarda usuários, tokens, logs e consentimentos
Código-fonte	Contém a lógica do sistema
Variáveis de ambiente	Guardam chaves e credenciais
Frontend	É a interface usada pelos usuários
Endpoints da API	São os pontos de entrada do sistema
Logs de auditoria	Registram ações importantes

Os endpoints mais importantes são:

- ``/auth/register``
- ``/auth/login``
- ``/auth/logout``
- ``/auth/2fa/setup``
- ``/auth/2fa/verify``
- ``/auth/forgot-password``
- ``/auth/reset-password``
- ``/user/me``
- ``/user/export-data``
- ``/user/delete-account``
- ``/admin/users-full``
- ``/admin/toggle-status/{user_id}``

Esses ativos precisam ser cuidados porque, se forem mal protegidos, podem permitir acesso indevido, perda de dados, alteração de informações ou indisponibilidade do sistema.

8. Identificação de ameaças e vulnerabilidades

Mesmo sendo um projeto acadêmico, é importante pensar nos riscos que podem afetar o sistema. Nesta seção, listamos ameaças comuns que podem atingir o JurisAI, principalmente na parte de autenticação e controle de usuários, mas também considerando o contexto jurídico e educacional da plataforma.

Ameaça	Explicação
Senha fraca	Usuários podem escolher senhas fáceis de adivinhar
Força bruta	Um atacante pode tentar várias senhas seguidas
Roubo de token	Um token salvo no navegador pode ser roubado em caso de falha no frontend
Acesso sem autorização	Um usuário comum pode tentar acessar rota de administrador
Vazamento de credenciais	Chaves ou senhas podem ser expostas se forem colocadas no código
Recuperação de senha insegura	Um fluxo fraco pode permitir troca indevida de senha
Injeção SQL	Entradas maliciosas podem tentar manipular consultas no banco
XSS	Código malicioso pode tentar rodar no navegador
CORS mal configurado	Sites externos podem tentar interagir com a API
Excesso de requisições	Muitas tentativas podem sobrecarregar a API
Acesso indevido a dados jurídicos futuros	Em versões com documentos e casos, um usuário não pode acessar informações de outro
Exposição de dados acadêmicos ou pessoais	Dados de perfil precisam ser tratados com cuidado
Uso incorreto da IA	Respostas geradas por IA precisam ser tratadas como apoio, não como decisão jurídica final

O projeto já possui algumas proteções, como validação de dados com Pydantic, hash de senha, bloqueio por tentativas erradas, autenticação em duas etapas, blacklist de token e verificação de administrador nas rotas sensíveis. Essas proteções mostram que a segurança foi pensada como parte do funcionamento do JurisAI, e não como um módulo separado sem relação com o produto.

Mesmo assim, identificamos melhorias importantes para versões futuras:

- Restringir o CORS em produção.
- Revisar o fluxo de redefinição de senha.
- Evitar salvar tokens no `localStorage` em uma versão final.
- Adicionar testes automatizados.
- Melhorar a documentação dos endpoints.
- Usar migrations para evolução do banco.

Essas observações não significam que o sistema está inutilizável, mas mostram que nós entendemos os pontos que precisam evoluir antes de uma publicação profissional.

9. Associação riscos e contramedidas

A tabela abaixo relaciona riscos identificados com as proteções que já existem ou que foram planejadas para o sistema.

Risco	Contramedida usada ou planejada	Situação
Senha em texto puro	Uso de hash com bcrypt	Implementado
Muitas tentativas de login	Bloqueio temporário após falhas	Implementado
Acesso a rotas protegidas	Validação de JWT	Implementado
Uso de token após logout	Blacklist de tokens	Implementado
Conta acessada sem segundo fator	2FA com TOTP	Implementado
Usuário comum acessando admin	Verificação de `is_admin`	Implementado
Dados pessoais sem controle	Consentimento e exportação LGPD	Implementado
Vazamento de segredo 2FA	Criptografia com Fernet	Implementado
Entrada de dados inválida	Validação com Pydantic	Implementado
CORS permissivo	Restringir origens em produção	Planejado
Reset de senha frágil	Revisar fluxo com token obrigatório	Planejado
Falta de testes automatizados	Criar suíte com pytest	Planejado

Essa matriz ajuda a mostrar que o projeto não foi desenvolvido apenas pensando nas telas, mas também no comportamento do sistema em situações de risco.

10. Testes de segurança realizados

Como o sistema ainda está em fase acadêmica, os testes foram organizados como uma combinação de validação manual, inspeção do código e planejamento de testes automatizados.

Os principais testes considerados foram:

- Cadastro com dados válidos.
- Cadastro sem aceite LGPD.
- Cadastro com senha fraca.
- Login com senha correta.
- Login com senha incorreta.
- Bloqueio após várias tentativas erradas.
- Acesso ao perfil com token válido.
- Acesso ao perfil sem token.
- Logout e tentativa de reutilizar token.
- Configuração de 2FA.
- Validação de código 2FA.
- Usuário comum tentando acessar rota administrativa.
- Exportação de dados do usuário.
- Exclusão de conta.

Também planejamos testes automatizados com **pytest**, principalmente para rotas protegidas, validação de senha, autenticação, permissões administrativas e recuperação de senha.

Para uma próxima entrega, a ideia é montar uma suíte de testes com banco de dados de teste, permitindo validar o funcionamento do sistema sem depender apenas de testes manuais no navegador.

10.1 Resultados dos testes

A tabela abaixo apresenta um relatório estruturado dos cenários avaliados ou planejados durante a validação do sistema.

Cenário	Resultado esperado	Status
Cadastro com LGPD aceito	Usuário criado com sucesso	Validado
Cadastro sem LGPD	Sistema deve recusar	Validado
Senha fraca	Sistema deve recusar	Validado
Login correto	Sistema retorna token	Validado
Login incorreto	Sistema retorna erro	Validado
Muitas falhas de login	Conta bloqueada temporariamente	Validado
Acesso sem token	Sistema nega acesso	Validado
Logout	Token é colocado na blacklist	Validado
Token após logout	Sistema deve recusar	Validado
Ativação de 2FA	Sistema gera QR Code e valida código	Validado
Acesso admin por usuário comum	Sistema deve negar	Validado
Exportação LGPD	Sistema retorna dados do usuário	Validado
Exclusão de conta	Sistema remove os dados do usuário	Validado
Reset de senha	Fluxo precisa de revisão final	Parcialmente validado

Exemplo de logs esperados:

INFO - Conta criada no sistema com aceite da LGPD.

INFO - Login realizado com sucesso.

WARNING - Falha de autenticação.

INFO - Conta bloqueada por excesso de tentativas.

INFO - Autenticação em duas etapas ativada.

INFO - Sessão encerrada com sucesso.

INFO - Exportação de dados solicitada.

Com base nessa análise, consideramos o sistema adequado para apresentação acadêmica e demonstração. Para uso em produção, ainda recomendamos ajustes no fluxo de recuperação de senha, CORS, testes automatizados e armazenamento do token no frontend.

Uso de artigos científicos e/ou normas técnicas

Mesmo usando uma linguagem mais simples, nós fundamentamos algumas decisões em referências conhecidas da área de segurança e desenvolvimento de software.

A LGPD foi usada como base para pensar no consentimento do usuário, finalidade do tratamento dos dados, exportação de informações e exclusão de conta.

A RFC 7519 foi usada como referência para o uso de JWT, que é o formato adotado para representar sessões autenticadas.

A RFC 6238 foi usada como referência para o TOTP, que é o padrão utilizado em aplicativos autenticadores para gerar códigos temporários.

A OWASP API Security Top 10 foi usada como guia para pensar em riscos comuns de APIs, como falhas de autenticação, autorização incorreta, excesso de requisições e configuração insegura.

A OWASP Password Storage Cheat Sheet foi usada como apoio para justificar o uso de bcrypt em vez de salvar senhas em texto puro ou usar hashes simples.

A ISO/IEC 27001 foi usada como referência geral para pensar em gestão de segurança da informação, principalmente na identificação de ativos, riscos e controles.

O NIST SP 800-63B foi usado como apoio para boas práticas de autenticação digital, autenticação multifator e gerenciamento de sessão.

Essas referências ajudaram a equipe a tomar decisões mais conscientes, mesmo sendo um projeto acadêmico.

Referências

CLARANET. NIST Cybersecurity Framework: entenda o que é e como aplicar. [S. l.]: Claranet, [202-]. Disponível em: <https://www.claranet.com/br/blog/nist-cybersecurity-framework>. Acesso em: 23 maio 2026.

IBM. O que é autenticação de dois fatores (2FA)? [S. l.]: IBM, [202-]. Disponível em: <https://www.ibm.com/br-pt/think/topics/2fa>. Acesso em: 23 maio 2026.

CUSTOMIZER M. API de recuperação e alteração de senha. [S. l.]: Customizer M, 2019. Disponível em: <https://customizerm.com.br/2019/09/23/api-de-recuperacao-e-alteracao-de-senha/>. Acesso em: 23 maio 2026.

AUTH0. Password reset. [S. l.]: Auth0, [202-]. Disponível em: <https://auth0.com/pt/learn/password-reset>. Acesso em: 23 maio 2026.